



Copyrighted material

This is an integral part of course **Next level Rust** and cannot be used in any context without explicit permission from **CLEUTON SAMPAIO DE MELO JUNIOR** (a.k.a. "the Author"). You can't copy, change, publish, or share with others.

Advanced ownership and lifetimes

Higher-Ranked Trait Bounds (HRTB)

In Rust every reference has a lifetime. A lifetime is the scope during which the reference is valid. Most of the time you do not need to write lifetimes because the compiler infers them.

Sometimes you need more than a single lifetime. You want to say:

This closure or type must work for any lifetime the caller provides

That is the role of Higher-Ranked Trait Bounds (HRTB). The syntax is:

```
for<'a> TraitBound
```

This means: for all possible lifetimes `'a`, this bound must hold. It is similar to generics over types, but here you are quantifying over lifetimes.

Example 1: No HRTB needed

When you pass a closure that only takes a reference, Rust already treats the lifetime in that parameter as higher-ranked. This allows the closure to be called with different lifetimes across calls without writing `for<'a>`.

```

//! ```cargo
//! [package]
//! edition = "2021"
//! ```

fn call_twice<F>(f: F) -> usize
where
    F: Fn(&str) -> usize,
{
    let s1 = "hello";           // &'static str
    let s2 = String::from("world"); // shorter lifetime
    f(s1) + f(&s2)
}

fn main() {
    println!("Result: {}", call_twice(|s| s.len()));
}

```

Explanation: For `Fn` traits, `Fn(&str)` is already shorthand for `for<'a> Fn(&'a str)`. That means the closure works for any `&str`. This is why one call with a `'static` string and another call with a short-lived string both compile.

Example 2: Fails without HRTB

If you define your own trait with a lifetime parameter, the compiler does not treat it as higher-ranked automatically. If you try to call it with two different lifetimes it fails.

```

//! ```cargo
//! [package]
//! edition = "2021"
//! ```

trait Visitor<'a> {
    fn visit(&mut self, s: &'a str);
}

#[derive(Default)]
struct Count(usize);

impl<'a> Visitor<'a> for Count {
    fn visit(&mut self, s: &'a str) {
        self.0 += s.len();
    }
}

// Without HRTB
fn run<'a, 'b, P>(p: &mut P, a: &'a str, b: &'b str)
where
    P: Visitor<'a>,

```

```
{
    p.visit(a); // ok
    p.visit(b); // error
}
```

```
fn main() {
    let mut c = Count::default();
    let s1 = String::from("hello");
    let s2 = String::from("world");
    run(&mut c, &s1, &s2);
}
```

```

Compiler message:

```
...
error: lifetime may not live long enough
argument requires that 'b must outlive 'a
```

```

Explanation: The bound `P: Visitor<'a>` says that `P` implements the `trait` `for` one lifetime `'a`. The second call needs `Visitor<'b>`, but the bound does not provide it.

Example 3: Fixed with HRTB

We fix the problem by making the bound universal. We say: `for any` lifetime `'x`, `P` implements `Visitor<'x>`.

```
```rust
//! ```cargo
//! [package]
//! edition = "2021"
//! ```

trait Visitor<'a> {
 fn visit(&mut self, s: &'a str);
}

#[derive(Default)]
struct Count(usize);

impl<'a> Visitor<'a> for Count {
 fn visit(&mut self, s: &'a str) {
 self.0 += s.len();
 }
}

fn run<'a, 'b, P>(p: &mut P, a: &'a str, b: &'b str)
where
 for<'x> P: Visitor<'x>,
{
```

```

 p.visit(a); // here 'x' = 'a'
 p.visit(b); // here 'x' = 'b'
}

fn main() {
 let mut c = Count::default();
 let s1 = String::from("hello");
 let s2 = String::from("world");
 run(&mut c, &s1, &s2);
 println!("total = {}", c.0);
}
...

```

Explanation: The bound ``for<'x> P: Visitor<'x>`` ensures that the `trait implementation` holds `for any` lifetime ``'x``. This allows the first call with ``'a`` and the second call with ``'b`` to both compile.

---

### ## Summary

- \* ``Fn(&str)`` already behaves as if it were ``for<'a> Fn(&'a str)`` which is why many closure examples compile without explicit HRTB.
- \* Your own traits with explicit lifetime parameters are not treated this way. If you want the same `type` to work with different lifetimes `in` the same scope you must add ``for<'a>``.
- \* HRTB is the tool that tells the compiler a `trait bound` is valid `for all` lifetimes, not just one.

This is why HRTB is important when designing traits and APIs that must remain flexible with references.